# Confidence-Gated Solver Budgeting for Kinematically Verified Hybrid Inverse Kinematics: A Two-Robot Evaluation with Three Training-Seed Sensitivity Runs

[AUTHOR INPUT NEEDED: full author names and ORCID identifiers]
[AUTHOR INPUT NEEDED: affiliations and corresponding-author email]

Pre-submission manuscript, 23 August 2026

**Evidence-status notice.** The six `paper_v2` runs are frozen formal tests. The runtime remediation in `latency_pilot_v3` is validation-only and is not presented as an independent test result. The original formal feasible-latency claim gate failed, deployment packaging is not yet complete for all six artifacts, and a fresh one-shot `test_v3` has not been started. This notice must be updated only after the preregistered release and test gates pass.

**Abstract**

Inverse kinematics (IK) for online robot control is not only a geometric root-finding problem: a runtime system must choose among solution basins, allocate a bounded numerical budget, reject futile requests, and verify every command before execution. We propose confidence-gated hybrid IK (CG-HIK), in which a five-member history-conditioned seed ensemble supplies candidate joint increments and a calibrated four-action classifier selects the entry point of a shared solver cascade (easy, medium, hard, or reject). Adaptive damped least squares, heterogeneous trust-region fallback seeds, and a common kinematic verifier remain responsible for refinement and acceptance. The primary comparison changes only the entry action: the fixed robust cascade always starts at the cheapest stage, whereas CG-HIK may skip work or reject without solving. A frozen protocol evaluated one robot-specific test set for Franka Panda and one for UR5e; each was reused across training seeds 17, 29, and 43 to measure model sensitivity rather than independent data replication. Each run contained 10,000 feasible point queries, 2,000 unreachable or constructed one-frame-inadmissible point queries, and forty 150-frame trajectories. Feasible-query success was unchanged in all six comparisons, while mean function evaluations fell by 28.21% for Panda and 33.74% for UR5e, averaged descriptively over the three seed runs. Rejectable-query P95 latency fell by 99.00% and 97.78%, respectively, and every accepted trajectory command satisfied the verifier's one-frame velocity inequality by construction. However, the eager learned-risk implementation increased feasible-query P95 latency by 35.70% and 60.28%, so the prespecified formal latency gate failed. A subsequent validation-only, numerically equivalent TorchScript export reduced proposed-method P95 latency versus the eager implementation by 45.17% and 41.73% and achieved proposed/fixed P95 ratios of 0.791 and 0.875. These optimized latency results remain subject to a new independent test, and packaging of all six deployment artifacts is incomplete. The evidence supports treating learned IK as a resource-allocation layer within a kinematically verified numerical system, rather than as a replacement for geometric solvers.

**Keywords:** inverse kinematics; manipulator control; hybrid solver; confidence calibration; adaptive computation; damped least squares; runtime verification; failure rejection

## 1 Introduction

Robot task planners, teleoperation interfaces, imitation-learning policies, and vision–language–action systems commonly specify motion in Cartesian task space, whereas a manipulator is actuated in joint space. Inverse kinematics therefore remains the interface between a transferable end-effector command and a robot-specific joint command. This interface becomes consequential in closed-loop control: a single discontinuous branch switch can create a joint-velocity spike, a near-singular update can amplify numerical error, and a slow failure on an unreachable request can consume an entire control period.

The forward map $x = f(q)$ is generally single-valued, but its inverse is better written as the set

$$\mathcal{S}(x) = \{q \in \mathcal{Q} : f(q) = x\}. \tag{1}$$

For a six-degree-of-freedom (DOF) arm, $\mathcal{S}(x)$ may contain several discrete branches; for a redundant arm, it can contain a continuous self-motion manifold. During online execution, the problem is not to solve one isolated

pose but to choose a sequence $q_{1:T}$ that is geometrically accurate, joint-limit admissible, and locally continuous. A useful abstraction is

$$q^{\star}_{1:T} = \arg\min_{q_{1:T}} \sum_{t=1}^{T} \lambda_p \ell_{\text{pose}}(q_t, x_t) + \lambda_s \ell_{\text{step}}(q_t, q_{t-1}) + \lambda_l \ell_{\text{limit}}(q_t), \tag{2}$$

subject to hard command-admissibility tests. Even when each target is reachable, the chosen numerical basin and the computation allocated to it determine whether the online sequence is executable.

Classical damped least-squares (DLS) methods stabilize Jacobian inversion near singularities [1–3], but they remain local and initialization-sensitive. Portfolio solvers such as TRAC-IK combine numerical strategies to reduce false negatives [4]; variable-step and restart strategies likewise show that fixed numerical policies are not optimal for every query [5]. At the same time, learned IK has progressed from single-output regression to generative solution-set modelling [6, 7], path-level generation [8], and history-conditioned joint-increment policies [9]. The most defensible role for learning is consequently not to certify a configuration, but to predict a useful solution basin and how much numerical work a query is likely to require.

Recent hybrid methods use learned approximations as warm starts followed by numerical refinement [10–12]. However, a typical pipeline still applies a fixed candidate count, iteration limit, and fallback policy to every query. This leaves three coupled questions unresolved: when is a learned seed trustworthy, how much solver budget should the query receive, and when should the system refuse rather than spend computation on a command that cannot pass the execution contract?

This paper studies those questions as runtime resource allocation. The contributions are:

1. a portfolio-specific four-action label (easy, medium, hard, reject) defined by the first solver stage that produces a kinematically verified command, rather than by arbitrary iteration bins or reachability alone;
2. a controlled primary comparison in which the proposed and fixed methods share the same seed ensemble, numerical stages, seed bank, fallback, and verifier, and differ only in their calibrated entry action;
3. a frozen two-robot, three-training-seed evaluation that reports feasible success, function evaluations (FEV), rejection, tail latency, trajectory completion, verifier interceptions, null ablations, and the failed formal latency claim gate;
4. a validation-only implementation study that identifies fixed risk-inference overhead, exports an exact TorchScript seed ensemble and vectorized frozen risk model, and checks routing and numerical equivalence; this is remediation evidence, not an independent optimized-latency result.

The central claim is deliberately bounded: learned models allocate computation; classical geometry refines and verifies the command. The present benchmark establishes this claim under exact URDF kinematics. It does not establish collision safety, physics-based controller tracking, acceleration or torque compliance, or physical robot reliability.

## 2 Related Work

### 2.1 Numerical IK, singularities, and solver portfolios

Jacobian inverse and pseudoinverse updates are efficient in well-conditioned local regions but can generate large joint velocities near rank loss. DLS reformulates the update to trade Cartesian residual against joint-step magnitude [1, 2]. Chiaverini et al. demonstrated variable damping, weighted DLS, online singular-value estimation, and feedback correction on an industrial manipulator [3]. These ideas remain essential numerical foundations, but damping does not identify a globally appropriate branch or guarantee recovery from a poor basin.

Practical solvers therefore introduce redundancy above the local update. TRAC-IK runs an improved Jacobian route and a sequential quadratic programming route concurrently [4]. IKSel retrieves and reorders seed configurations and reselects distant seeds after failure [13]. RelaxedIK formulates IK as real-time motion synthesis with pose, smoothness, collision, singularity, and continuity terms [14]. Modern geometric decomposition can also return all branches for broad classes of 6R arms with very low numerical overhead [15], whereas viability-aware QP formulations can directly encode joint range, velocity, acceleration, and collision constraints [16]. These are important capabilities beyond the exact-kinematics portfolio evaluated here. Collectively, the methods motivate a solver-system view, but they do not learn a calibrated query-specific entry action over a fixed, auditable portfolio.

### 2.2 Learning solution sets and trajectories

A pose-only regression $x \mapsto q$ is mismatched to a one-to-many inverse. IKFlow uses conditional normalizing flows to sample diverse configurations and can hand candidates to a numerical solver for refinement [6]. GGIK encodes manipulators as distance-geometric graphs, enabling multi-solution generation and cross-robot generalization [7]. CppFlow lifts candidate generation to Cartesian paths, coupling a generative model to path-level search and numerical optimization [8]. These works primarily improve proposal coverage.

History conditioning localizes branch choice. Predicting $\Delta q_t$ from the current state and target pose turns a global inverse into a local motion decision. MimicIK uses this formulation with conditional flow matching and FK consistency on teleoperation data [9]; because it is currently a preprint, its deployment claims are treated as a trend signal rather than settled evidence. In our system, history conditioning is used for proposals, while a separate learned action model predicts the required numerical stage and an explicit verifier retains the acceptance decision.

### 2.3 Hybrid learning and numerical refinement

CycleIK combines neural prediction, FK-aware objectives, and optional SLSQP or genetic refinement [10]. Recent engineering studies similarly position soft-computing or graph-network predictions as initializers for Jacobian-based methods [11, 12]. These systems support the division of labor used here: data-driven models provide a global prior, and numerical geometry supplies precision. The present work differs by learning an action over a shared cascade, enforcing data-split separation for calibration and policy selection, comparing against an identical fixed-entry cascade, and retaining failure rejection as a first-class runtime outcome.

### 2.4 Runtime confidence and failure detection

Calibration turns classifier scores into in-distribution action probabilities, but it does not by itself establish out-of-distribution (OOD) awareness. In manipulation policy deployment, runtime failure detectors have combined epistemic signals, sequential OOD detection, and conformal calibration to decide when a policy should be escalated [17]. Our present confidence values are narrower: they are isotonic-calibrated probabilities for four portfolio actions on the declared synthetic distribution. They are used to choose a solver entry or a zero-solve refusal; no OOD detector or model-deferral action is evaluated. A two-sided policy that separates high-confidence command rejection from uncertainty-triggered solver escalation is therefore future work, not a contribution claimed by this experiment.

Table 1 separates the present contribution from adjacent IK capabilities. The table describes functionality rather than empirical superiority, because IK-Geo, viability QP, generative path IK, and CG-HIK solve different problem contracts.

**Table 1:** Functional positioning relative to adjacent IK paradigms. A check mark denotes an explicit method capability, not a result from the present benchmark.

| Paradigm | Multiple solutions | Numerical refinement | Query-adaptive entry | Explicit refusal | OOD deferral | Acceptance scope |
|---|---|---|---|---|---|---|
| IK-Geo [15] | ✓ | – | – | – | – | geometric IK |
| Viability QP [16] | – | ✓ | – | infeasibility | – | encoded state/collision constraints |
| IKFlow/CppFlow [6, 8] | ✓ | ✓ | – | – | – | pose/path constraints |
| CG-HIK (this work) | candidate set | ✓ | ✓ | ✓ | – | pose, limits, one-frame velocity |

## 3 Materials and Methods

### 3.1 Online IK query and acceptance contract

At frame $t$, a query is

$$\xi_t = (x_t^d, q_{t-1}, \Delta t), \qquad x_t^d \in SE(3), \tag{3}$$

with $\Delta t = 0.02$ s. A candidate $q_t$ is accepted only if it is finite and

$$\|p(q_t) - p_t^d\|_2 \le 1 \text{ mm}, \tag{4}$$

$$\theta(R(q_t)^\top R_t^d) \le 0.5^\circ, \tag{5}$$

$$q_{\min} \le q_t \le q_{\max}, \tag{6}$$

$$|q_t - q_{t-1}| \le \dot{q}_{\max}\Delta t + 10^{-4}. \tag{7}$$

The last inequality is evaluated with the robot's joint-difference convention. A numerical solver's convergence flag is insufficient; all four conditions are checked by the common verifier before a configuration can be returned.

### 3.2 History-conditioned candidate proposal

The proposal network consumes the normalized previous joint state, target position, and a continuous 6D rotation representation:

$$z_t = [\bar{q}_{t-1}, p_t^d, \rho_6(R_t^d)]. \tag{8}$$

Five multilayer perceptrons (three hidden layers of 256 units) predict joint increments. For member $m = 1, \ldots, 5$,

$$\Delta\hat{q}_t^{(m)} \in [-0.25, 0.25]^n. \tag{9}$$

Each member is trained on a bootstrap resample with a Huber joint-increment loss and forward-kinematics position and orientation penalties. Deep-ensemble disagreement provides mean and maximum uncertainty features [18]. Candidates are clipped to the URDF limits, scored by pose residual, joint change, and joint-limit margin, and deduplicated at a maximum joint-distance threshold of 0.05 rad.

The proposal is not accepted directly. Its best candidate and ensemble diagnostics are inputs to the action model, and every numerical result is checked by the same verifier.

### 3.3 Portfolio-specific action labels and calibrated risk model

For a known-feasible, one-frame-continuous query, the oracle label is the first stage of the locked cascade that produces a kinematically verified configuration. Queries that are generated as outside a conservative radial reach bound or are designated one-frame-inadmissible are labelled reject without spending solver work during label creation. If all stages fail on a nominally feasible query, the label is also reject. Thus *easy*, *medium*, and *hard* name entry levels of this locked portfolio; they are not intrinsic categories of IK difficulty. Likewise, *reject* means "do not invoke this portfolio for this generated request", not a general proof of physical or collision-aware unreachability.

The nine-dimensional risk feature vector is

$$r_t = [e_p, e_R, u_{\text{mean}}, u_{\max}, \sigma_{\min}, m_q, \|\Delta q\|_2, d_p, d_R], \tag{10}$$

where $e_p, e_R$ are the best seed's pose errors, $u$ denotes ensemble disagreement, $\sigma_{\min}$ is the candidate Jacobian's minimum singular value, $m_q$ is its minimum normalized joint margin, $\Delta q$ is its joint change, and $d_p, d_R$ are Cartesian step distances from the previous state.

An MLP classifier and a histogram gradient-boosting classifier were fitted on `risk_train`; model family was selected by negative log likelihood on `risk_validation`. The selected four-class probabilities were independently isotonic-calibrated on `calibration` and renormalized. Throughout this paper, *confidence* means these in-distribution calibrated action probabilities; it is not an OOD guarantee. Calibration is relevant because routing thresholds consume probability values, not only class ranks [19]. The action thresholds were selected once on a dedicated `policy_validation` split, subject to false rejection no greater than 1% and reject recall at least 95%, before either classifier-test or runtime-test evaluation.

With calibrated probabilities $P_E, P_H, P_R$, the gate is

$$a(r_t) = \begin{cases} \text{reject}, & P_R \geq \tau_R, \\ \text{easy}, & P_E \geq \tau_E, \\ \text{hard}, & P_H \geq \tau_H \text{ or hard is the largest non-reject class}, \\ \text{medium}, & \text{otherwise}. \end{cases} \tag{11}$$

The threshold grid was frozen before test. No threshold reported here was modified in response to formal test or latency-pilot outcomes.

### 3.4 Shared numerical cascade

Table 2 defines the locked entry actions. The fixed robust baseline always enters at easy and escalates on failure. The proposed method enters at the predicted action; all later stages, budgets, candidates, fallback solvers, and verification logic are identical. This construction isolates work skipped by routing.

**Table 2:** Locked action and escalation contract.

| Action | First numerical stage | Initial budget | Escalation after failure |
|---|---|---|---|
| Easy | Previous-state adaptive DLS | 1 iteration | Medium, then hard |
| Medium | Best history-conditioned learned seed with DLS | 1 iteration | Hard |
| Hard | Learned and previous seeds with DLS, followed by heterogeneous TRF seeds | 25 iterations per DLS seed | Bounded fallback only |
| Reject | No numerical solve | 0 | None |

At a DLS iteration, the weighted update is

$$\Delta q = J_w^\top (J_w J_w^\top + \lambda^2 I)^{-1} e_w. \tag{12}$$

The damping is a smooth function of the minimum Jacobian singular value,

$$\lambda(\sigma) = \lambda_{\min} + (\lambda_{\max} - \lambda_{\min}) \left(1 - \text{clip}\left(\frac{\sigma}{\sigma_0}, 0, 1\right)\right)^2, \tag{13}$$

with $\lambda_{\min} = 10^{-4}$, $\lambda_{\max} = 0.1$, and $\sigma_0 = 0.05$. Backtracking scales $(1, 0.5, 0.25, 0.1)$ are tried and the per-joint update is clipped at 0.1 rad. Adaptive damping is a numerical foundation rather than a claimed novelty.

The hard stage attempts DLS from the best learned seed and previous state, then uses a bounded trust-region reflective (TRF) solver from previous, learned, and up to two KD-tree-retrieved seeds. The TRF budget is 100 function evaluations. Only a verifier-accepted result ends the cascade.

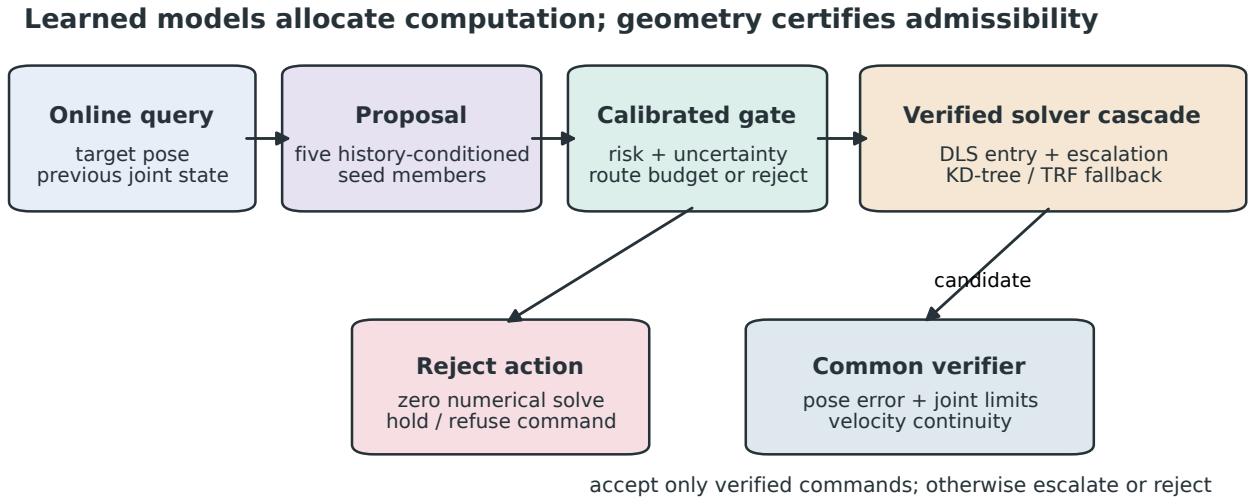


**Figure 1:** Architecture of CG-HIK. The proposal and calibrated gate allocate computation; the numerical cascade generates configurations, and the common verifier alone admits a command. Reject is an explicit zero-solve action.

### 3.5 Exact-export latency remediation

Formal `paper_v2` used the original eager inference path. After its outputs and claims were frozen, a separate validation-only pilot decomposed per-query latency into feature preparation, NumPy/PyTorch conversion, seed inference, uncertainty/risk inference, routing, numerical solve, verification, serialization, and total end-to-end time. Timing used `perf_counter_ns`; warmup preceded measurement; baseline and proposed queries were paired and interleaved; disk writing was excluded from timed intervals.

Implementation changes did not alter the algorithm: models were set to evaluation and inference mode, contiguous float32 seed inputs were preallocated, static normalization and robot features were cached, repeated geometry was removed, and the five frozen members were executed in one exact TorchScript call. The histogram gradient-boosting trees and isotonic calibrators were exported as frozen numerical parameters and evaluated without per-tree Python dispatch. The selected backend was `torchscript_exact`; a single-member path was diagnostic only and was ineligible for selection.

Numerical equivalence required maximum seed, probability, and risk-score errors within the frozen tolerances; 100% routing-action, accepted-command, fallback, FEV, and verification-reason agreement; and unchanged success, rejection, calibration, and trajectory metrics. This pilot selected an implementation, not an algorithm or threshold.

## 4 Experimental Design

### 4.1 Robots, software, and data separation

Experiments used exact URDF kinematics for a 7-DOF Franka Panda and a 6-DOF UR5e. FK, Jacobians, and URDF limits were evaluated with Pinocchio [20]; neural models used PyTorch [21]; the risk model used scikit-learn [22]; TRF and supporting numerical routines used SciPy [23]. Formal tests ran on Linux with Python 3.12.13, NumPy 2.4.4, SciPy 1.17.0, scikit-learn 1.9.0, Pinocchio 3.9.0, PyTorch 2.11.0+cu128, CUDA 12.8, and an NVIDIA Quadro RTX 8000.

Each robot and training seed used the split roles in Table 3. The runtime test was independently generated and locked; the same test queries were reused across training seeds within each robot so seed differences measure model sensitivity, not three independent test datasets.

**Table 3:** Data split roles and sizes per robot and training seed.

| Split | Samples | Sole permitted use |
|---|---|---|
| Seed train | 200,000 | Fit five seed members |
| Seed validation | 20,000 | Seed diagnostics and trajectory pilot source |
| Risk train | 60,000 | Fit action classifier |
| Risk validation | 15,000 | Select classifier family |
| Calibration | 15,000 | Fit isotonic calibration |
| Policy validation | 15,000 | Select action thresholds/guard quantile |
| Risk test | 20,000 | One-time action and calibration metrics |
| Runtime test points | 12,000 | One-time solver evaluation |
| Runtime test trajectories | $40 \times 150$ | Closed-loop kinematic sequences |

The experiment contains four evidence layers that must not be pooled (Table 4). In particular, favorable optimized latency comes from a validation pilot, not from the formal eager test. The subsequent release attempt was a packaging/equivalence gate rather than a performance experiment: it stopped at UR5e/seed 43 because frozen offline probability and risk-score discrepancies of $1.26 \times 10^{-12}$ and $1.14 \times 10^{-12}$ exceeded the locked $10^{-12}$ limits, although sampled runtime actions and outcomes agreed exactly. It generated no `test_v3` evidence.

**Table 4:** Evidence provenance and permitted interpretation.

| Evidence layer | Robots/seeds | Split | Permitted use |
|---|---|---|---|
| `paper_v2` formal | both / 17, 29, 43 | locked test | success, FEV, rejection, eager latency, and descriptive paths |
| `latency_pilot_v3` | both / 17 | validation only | profile and select an equivalent inference implementation |
| `release_v3_locked` | stopped at UR5e/43 | no new test | deployment-equivalence diagnostic only |
| `test_v3` | absent | absent | no optimized independent claim |

Point tests comprised 5,000 local in-distribution queries, 1,000 each near singularity, near a joint limit, and near the sampled workspace boundary, 2,000 hard-valid queries, 1,000 constructed large-step queries, and 1,000 radially unreachable queries. For each large-step case, the previous and target configurations were sampled independently inside a 10% joint-limit margin; target FK made the pose reachable, but the record was designated one-frame-inadmissible by construction. This does not prove that no alternate IK branch lies inside the velocity bound. For each unreachable case, the target radius was sampled uniformly from $[r_{\mathrm{ub}} + 0.05, r_{\mathrm{ub}} + 0.25]$ m, where $r_{\mathrm{ub}}$ is the sum of kinematic-chain joint-origin translation norms plus all prismatic travel. The two rejectable strata are therefore controlled stress tests with generation cues, not an estimate of natural failures or OOD robustness. Hard-valid queries were screened using only the previous-state numerical solver: they failed the one-iteration easy stage and passed the 25-iteration stage while retaining a known velocity-feasible reference. Trajectories comprised ten paths from each of four families (smooth multi-joint, wrist/orientation, low-manipulability, and joint-limit-skimming), with 150 frames per path. Targets were obtained by FK from independently generated velocity-feasible references; tested solvers were not used to filter the trajectories.

## 4.2 Comparators and ablations

The primary pair was `proposed_v2` and `fixed_robust_cascade`. Four additional primary comparators were conventional previous-state DLS, learned-seed DLS, a validation-constrained Cartesian-step threshold guard attached to the same cascade, and previous-state TRF. Seven diagnostic ablations removed history, retained one ensemble member, removed disagreement features, removed calibration, disabled explicit rejection, disabled fallback, or replaced adaptive with fixed damping. Thus the full benchmark contained six main methods and seven ablations, not thirteen unrelated learned models.

The threshold guard's quantile was chosen on `policy_validation` under the same false-reject and reject-recall constraints as the learned gate. It provides an interpretable comparison for the question of whether Cartesian step magnitude alone is sufficient for resource allocation.

### 4.3 Outcomes and statistical analysis

Primary point outcomes were kinematically verified success on known-feasible queries, mean function evaluations (FEV) on those queries, correct rejection and FEV on rejectable queries, and whole-trajectory completion. In DLS, one FEV is each call to FK at the current iterate, a backtracking trial, or the terminal check; Jacobian/SVD work, candidate scoring, risk inference, and final verification are excluded. For TRF, FEV is SciPy's residual-call counter `nfev`; its post-solve pose check is excluded. A query's FEV is summed over every executed solver attempt. This counter is therefore a portfolio-specific numerical-work proxy, not a hardware-independent FLOP count.

Tail latency was stratified into feasible and rejectable point queries. Secondary outcomes included hard-valid success, P99 latency, fallback use, verifier interceptions, pose error, frame acceptance, and command-spike rate. For an accepted trajectory frame, *spike* is $\mathbb{1}[\exists j : |q_{t,j} - q_{t-1,j}| > \dot{q}_{\max,j}\Delta t + 10^{-4}]$; the reported rate uses all trajectory frames as its denominator and rejected frames contribute false. Because this is exactly the verifier's velocity inequality, a zero accepted-command spike rate is an acceptance invariant, not independent evidence of acceleration, jerk, or perceptual smoothness.

The point query (with its generating identifier) was the independent unit for point estimands; the complete path was the independent unit for trajectory completion. Frames after a tracking failure were not pooled into point success. Paired 95% confidence intervals used 10,000 cluster-bootstrap resamples. The generating trajectory identifier defined a cluster; it is unique for point queries and shared by all frames of a path. Cluster effects were proposed-minus-fixed means. The bootstrap resampled clusters with replacement, and the two-sided sign-flip test randomly multiplied each cluster effect by $-1$ or $+1$; its Monte Carlo value used the finite-sampling correction $(b + 1)/(10{,}000 + 1)$. Holm's step-down procedure corrected the three prespecified point comparisons: feasible success, feasible FEV, and rejectable acceptance. Exact adjusted $p$ values are reported where inference was prespecified. Training seed is a sensitivity replicate; no inferential test treats the three seeds as three independent datasets. Latency summaries are empirical batch-one P50/P95/P99 statistics and are not averaged into a query-level hypothesis test. Formal timing used 200 warmup iterations, five repeats per method/query with the median retained, deterministic per-query method-order randomization, one configured CPU thread, and CUDA synchronization around each solve. The 20 ms value defines the kinematic velocity contract; it is not a measured hard wall-clock deadline. Deadline-miss frequency, CPU affinity, and real-time scheduling were not prespecified outcomes.

The formal paper-level gate required the same effect direction on both robots and all three seeds, plus prespecified success, rejection, risk, trajectory, and latency limits. A failed threshold was reported without test-set retuning. The validation-only runtime pilot used 750 paired feasible queries per robot for selected-backend latency, plus locked category and trajectory subsets for equivalence and outcome gates.

## 5 Results

### 5.1 Held-out action risk was well separated and calibrated

Across Panda seeds, reject AUROC ranged from 0.9974 to 0.9980, reject expected calibration error (ECE) ranged from $9.06 \times 10^{-4}$ to $1.23 \times 10^{-3}$, and reject recall ranged from 0.9889 to 0.9892. UR5e reject AUROC exceeded 0.99999995 in all three runs, ECE ranged from $2.36 \times 10^{-5}$ to $5.88 \times 10^{-5}$, and reject recall was 0.99973. Held-out false-reject rate was zero in five runs and $6.15 \times 10^{-5}$ in UR5e seed 29. These metrics passed the frozen risk gates, but non-reject macro-F1 remained approximately 0.59–0.64, showing that fine-grained effort prediction was materially harder than reject detection.

The proposed runtime did not rely on the classifier's class decision as proof of an IK solution. Non-reject actions entered the shared numerical cascade, and the common verifier retained the final decision.

### 5.2 The gate preserved feasible success and reduced numerical work

Figure 2a–b reports the six primary paired comparisons. Panda feasible success was 99.99% for both methods in every run; UR5e feasible success was 100% for both methods in every run. The proposed-minus-fixed success difference was therefore 0 percentage points in all six runs, within the prespecified $-1$ percentage-point margin.

Mean FEV fell in every run. Panda per-run reductions were 28.50%, 28.04%, and 28.08% (descriptive mean 28.21%); UR5e reductions were 33.54%, 33.77%, and 33.92% (descriptive mean 33.74%). The paired mean FEV differences ranged from $-1.506$ to $-1.469$ evaluations for Panda and $-2.015$ to $-1.993$ for UR5e. Every cluster-bootstrap interval excluded zero; the Holm-adjusted sign-flip value was $p = 0.000300$ in each run. Table 5 summarizes the descriptive three-seed means without treating them as independent datasets.

**Table 5:** Primary formal-test outcomes. Values are descriptive means over three training-seed runs on the same locked test set within each robot. FEV is function evaluations. Each P95 entry is the mean of three per-run percentiles, not a pooled percentile.

| Robot | Method | Feas. success (%) | Feas. FEV | Feas. P95 (ms) | Reject FEV | Reject P95 (ms) | Traj. completion (%) |
|---|---|---|---|---|---|---|---|
| Panda | Fixed | 99.99 | 5.265 | 3.447 | 393.680 | 354.315 | 100.00 |
| Panda | Proposed | 99.99 | 3.780 | 4.679 | 0.053 | 3.539 | 100.00 |
| UR5e | Fixed | 100.00 | 5.942 | 3.153 | 218.183 | 158.305 | 96.67 |
| UR5e | Proposed | 100.00 | 3.937 | 5.055 | 0.000 | 3.516 | 97.50 |

## 5.3 Declared comparators bound the originality claim

Table 6 makes all six declared methods visible under the same frozen eager timing protocol. Previous-state DLS had the lowest feasible P95, but did not match the shared cascade's success on the full feasible mixture for either robot and completed only 95% of UR5e paths. Previous-state TRF was especially weak on Panda. The Cartesian-step guard rejected the constructed stress strata cheaply, but its feasible success was 0.92 and 0.87 percentage points below CG-HIK on Panda and UR5e; on hard-valid queries the gaps were 4.58 and 4.12 points. Thus the evidence supports a portfolio-specific trade-off: learned routing retained the robust cascade's feasible success while skipping numerical work. It does not support a claim that the eager implementation is the fastest IK method overall.

**Table 6:** Declared formal-test comparators. Values are descriptive means over three training-seed sensitivity runs on one locked test set per robot. P95 is batch-one eager latency; all methods rejected all constructed rejectable queries.

| Robot | Method | Feas. success (%) | Hard-valid success (%) | Feas. FEV | Feas. P95 (ms) | Traj. completion (%) |
|---|---|---|---|---|---|---|
| Panda | Previous-state DLS | 99.667 | 100.000 | 3.428 | 1.201 | 100.0 |
| Panda | Previous-state TRF | 84.650 | 57.100 | 77.370 | 86.997 | 85.0 |
| Panda | Learned seed + fixed DLS | 99.253 | 97.800 | 3.393 | 2.219 | 100.0 |
| Panda | Cartesian-step guard | 99.073 | 95.417 | 5.146 | 3.445 | 100.0 |
| Panda | Fixed robust cascade | 99.990 | 100.000 | 5.265 | 3.447 | 100.0 |
| Panda | CG-HIK | 99.990 | 100.000 | 3.780 | 4.679 | 100.0 |
| UR5e | Previous-state DLS | 99.853 | 100.000 | 3.617 | 1.092 | 95.0 |
| UR5e | Previous-state TRF | 99.810 | 99.200 | 5.191 | 4.717 | 100.0 |
| UR5e | Learned seed + fixed DLS | 99.820 | 99.700 | 3.591 | 2.040 | 97.5 |
| UR5e | Cartesian-step guard | 99.127 | 95.883 | 5.827 | 3.156 | 96.7 |
| UR5e | Fixed robust cascade | 100.000 | 100.000 | 5.942 | 3.153 | 96.7 |
| UR5e | CG-HIK | 100.000 | 100.000 | 3.937 | 5.055 | 97.5 |

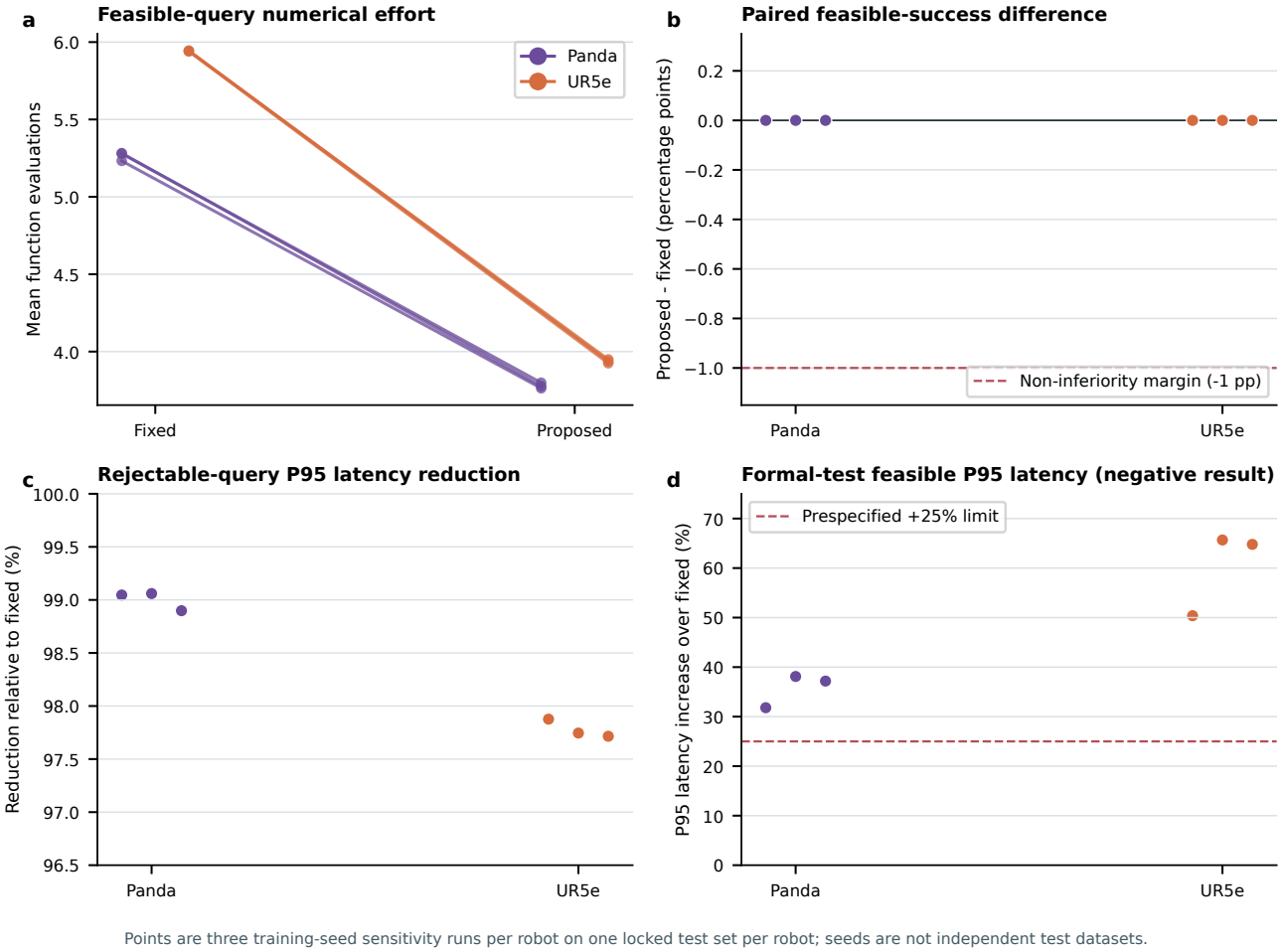


**Figure 2:** Frozen formal-test outcomes. (**a**) Mean function evaluations on 10,000 feasible point queries per run. Each line is one training seed and is paired within robot. (**b**) Proposed-minus-fixed feasible-success difference; the dashed line is the prespecified non-inferiority margin. (**c**) Rejectable-query P95 latency reduction for 2,000 rejectable points per run. (**d**) The negative formal result: eager proposed inference exceeded the allowed feasible P95 overhead in all six runs. Three seeds share one locked test set per robot and are sensitivity runs, not three independent test datasets.

## 5.4 Explicit rejection removed futile work and reduced verifier load

Both fixed and proposed systems rejected every radially unreachable or constructed one-frame-inadmissible test query, so rejectable acceptance difference was zero. The difference was computational: the fixed cascade attempted the full path from its easy entry, whereas the proposed gate usually issued reject with zero solver calls. Mean rejectable-query FEV fell by 100% in all UR5e runs and by 100%, 100%, and 99.96% in Panda. Rejectable P95 latency fell by 98.90–99.06% for Panda and 97.72–97.88% for UR5e (Figure 2c).

The verifier also exposed why convergence alone is an unsafe acceptance rule under the defined command contract. The fixed cascade produced 933–944 verifier interceptions per Panda run and 954–961 per UR5e run. The proposed system reduced these counts to 1–2 for Panda and zero for UR5e, primarily by not executing predictably futile large-step actions. These are kinematic contract interceptions, not collision or torque safety claims.

## 5.5 Trajectory outcomes were retained

All Panda fixed and proposed runs completed all forty trajectories. UR5e fixed completion was 95.0%, 97.5%, and 97.5% across seeds; proposed completion was 97.5% in all three, yielding one +2.5 percentage-point difference and two ties. The recorded command-spike rate was zero throughout, as required for accepted frames by the shared velocity verifier. Because the complete path was the independent unit, the 6,000 trajectory frames per run were not pooled into the 10,000-point feasible-success estimand.

These forty-path results per robot are descriptive; no path-level equivalence or non-inferiority claim is made. They document closed-loop completion under the tested URDF and per-frame velocity contract. They do not establish low-level controller tracking, acceleration/jerk feasibility, perceptual smoothness, or hardware performance.

## 5.6 Ablations identified one strong mechanism and several null components

Figure 3 retains the null and negative ablations rather than selecting only supportive results. Disabling explicit rejection restored most rejectable-query work: descriptive mean FEV was 388.5 for Panda and 213.5 for UR5e, close to the fixed cascade's 393.7 and 218.2. This confirms that the reject action, rather than faster failure inside the numerical solver, produced the rejectable-query saving.

Disabling fallback reduced descriptive feasible success from 99.99% to 99.71% for Panda and from 100% to 99.88% for UR5e, indicating a small but operational recovery role. Fixed damping was strongly unfavorable for Panda (99.13% feasible success and 7.03 mean FEV) but had lower feasible FEV on UR5e; adaptive damping is therefore retained as a robust shared foundation, not claimed as uniformly optimal.

Removing history, retaining one member, removing uncertainty features, or removing calibration changed formal operational FEV only slightly. These ablations do not support a claim that each learned component is individually necessary for the tested runtime outcomes. The ensemble remains part of the frozen system because uncertainty and calibration were preregistered design components, but the paper makes no diversity or causal-necessity claim from these null comparisons.

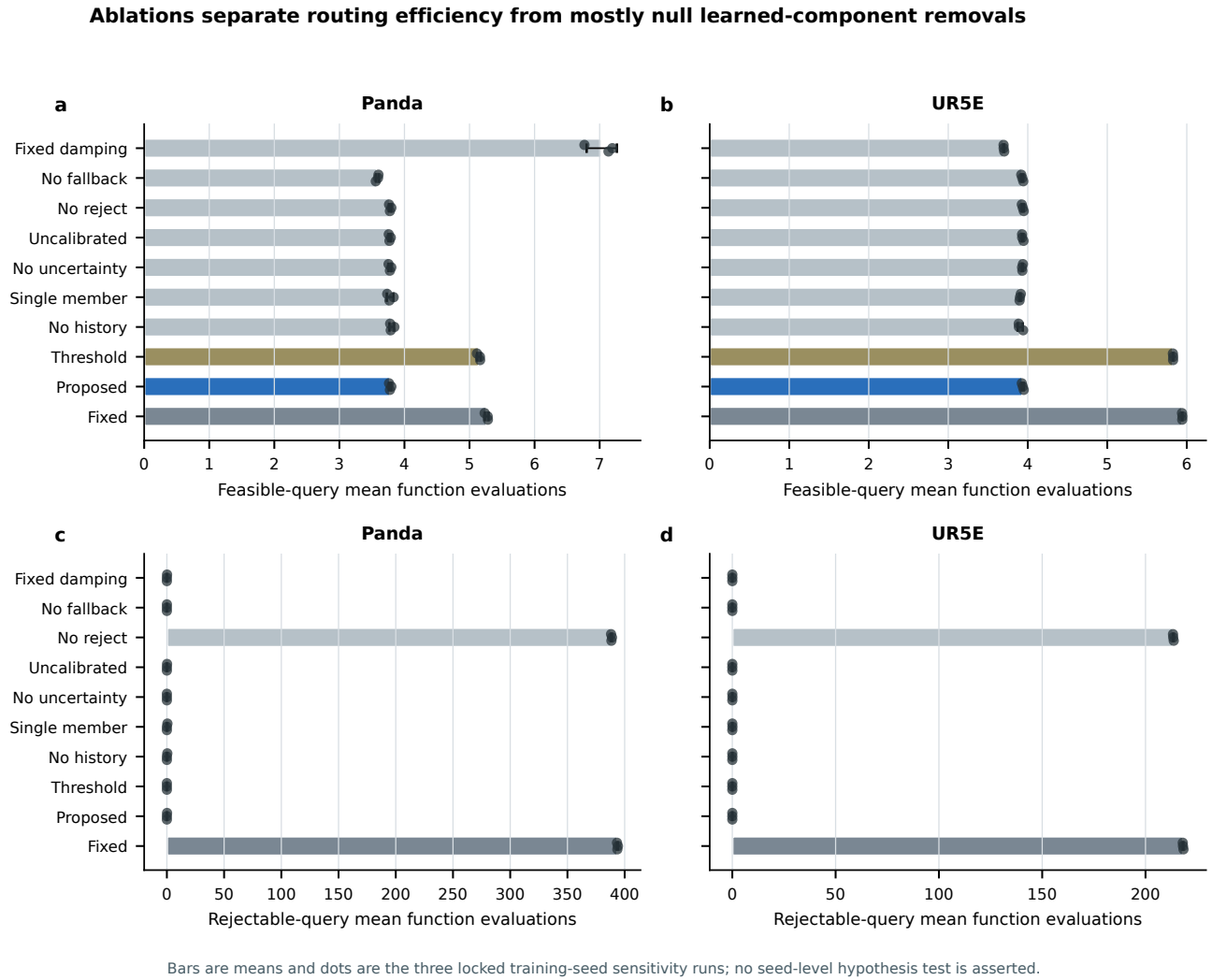


**Figure 3:** Diagnostic ablations. (**a,b**) Feasible-query mean FEV for Panda and UR5e. (**c,d**) Rejectable-query mean FEV. Bars show the descriptive mean plus or minus one seed standard deviation; dots show all three training-seed sensitivity runs. The no-reject ablation restores futile rejectable-query work, whereas several learned component removals are operationally close to the proposed method. No seed-level hypothesis test is asserted.

## 5.7 The original formal feasible-latency claim failed

The formal test revealed a consistent negative result. Although numerical effort fell, eager learned inference added an approximately fixed per-query cost. Feasible P95 latency increased in all six runs: by 31.8–38.1% for Panda and 50.4–65.7% for UR5e, exceeding the prespecified +25% limit (Figure 2d). The paper-level gate was therefore false. Thresholds, labels, risk rules, solver budgets, and claim limits were not changed after observing the failure.

This finding separates algorithmic and implementation efficiency. The learned gate correctly avoided numerical evaluations, but the original histogram-gradient-boosting inference path dispatched hundreds of trees per sample in Python. Stage profiling on validation identified uncertainty/risk inference as the largest eager fixed stage at approximately 1.96–2.00 ms P95. Consequently, an optimized inference implementation was investigated without altering routing semantics.

## 5.8 Validation-only exact export met the runtime pilot gates

Figure 4 reports the implementation-only validation pilot. With the exact TorchScript backend, feasible proposed/fixed P95 ratios were 0.791 for Panda and 0.875 for UR5e, both below the locked 1.15 target. Proposed P95 latency was 2.656 ms versus 3.358 ms fixed for Panda, and 2.705 ms versus 3.090 ms fixed for UR5e. Relative to the eager proposed implementation, P95 fell by 45.17% and 41.73%, exceeding the 30% target.

The exact backend achieved maximum risk-probability error below $4 \times 10^{-13}$, zero seed-output error under the stored comparison, 100% route, acceptance, fallback, FEV, and verification-reason agreement, and unchanged success, rejection, AUROC, ECE, trajectory completion, and command-spike gates. After optimization, numerical solving rather than learned risk inference was the largest P95 stage. A Panda outlier reached 306.46 ms because the solver consumed 373 function evaluations; it did not affect P95 but demonstrates that rare numerical tails remain a separate runtime concern.

These results are validation-only and used training seed 17. They establish that an equivalent low-overhead implementation exists, but the subsequent locked release did not package all six artifacts under its strict offline equivalence tolerance. They do not independently confirm the optimized latency claim. A completed unchanged release and fresh preregistered `test_v3` are required before replacing the formal latency conclusion.

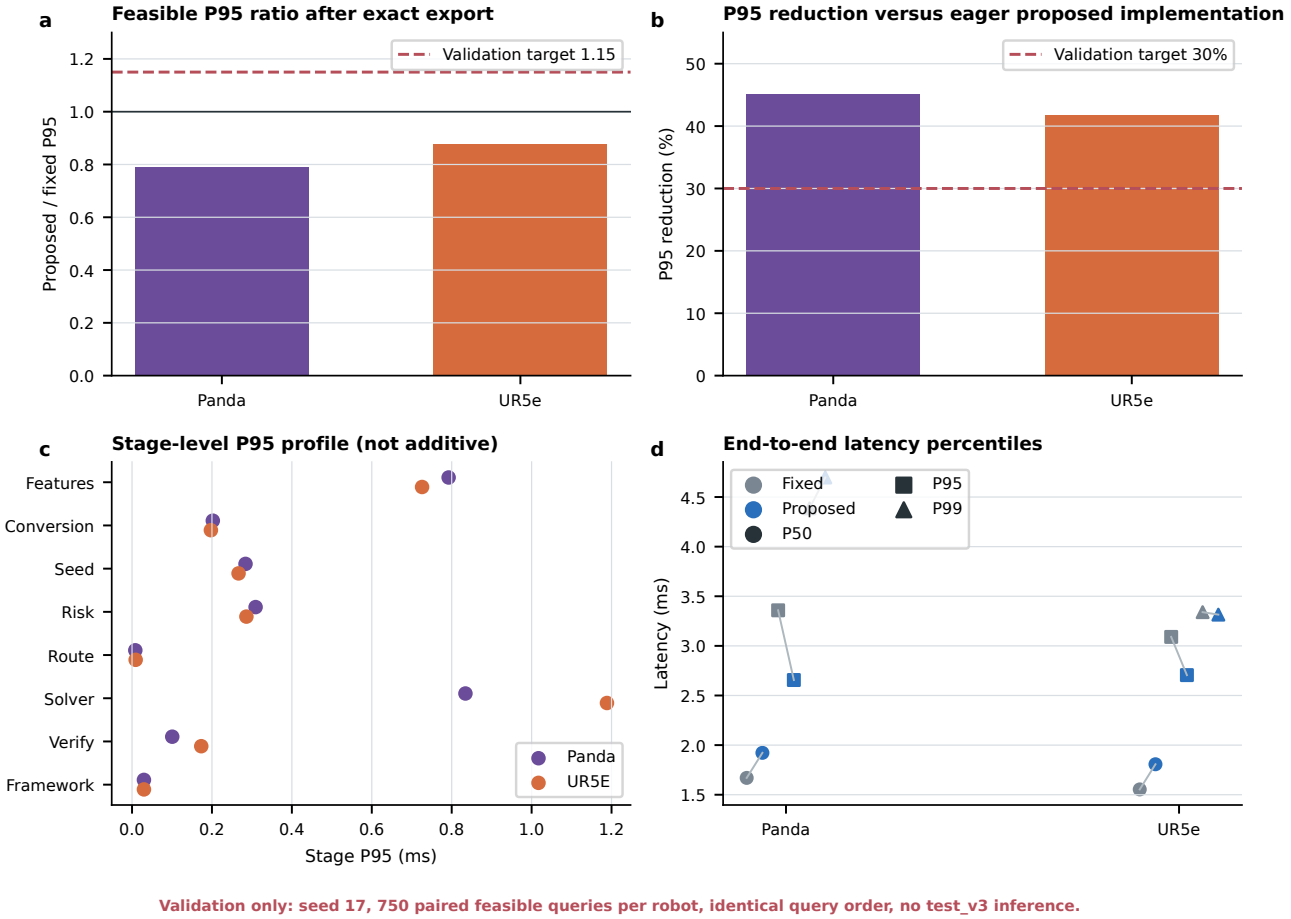


**Figure 4:** Validation-only latency remediation using the selected exact TorchScript backend. (**a**) Proposed/fixed feasible P95 ratio; lower than 1.15 passes the pilot target. (**b**) reduction relative to the eager proposed implementation; both robots exceed 30%. (**c**) stage-level P95 values, which are not additive. (**d**) paired end-to-end P50, P95, and P99. Each robot contains 750 paired feasible validation queries at training seed 17. No `test_v3` inference is made.

# 6 Discussion

## 6.1 IK as kinematically verified resource allocation

The results support a system-level interpretation of modern IK. The proposal stage locates candidate basins; the calibrated gate predicts a portfolio entry or refusal; the numerical cascade restores pose accuracy; and the verifier enforces a non-learned acceptance contract. No single module solves the full online problem. This division of labor is consistent with generative warm-start systems [6, 8, 10] and recent engineering hybrids [11, 12], but the action-gated formulation makes computation itself an explicit predicted variable.

The fixed robust comparison is important. If the proposed method had been compared only with an unrelated DLS implementation, FEV savings could have resulted from different fallback availability, seed banks, or verification rules. Here the primary pair shares the entire portfolio and differs only in entry action. Therefore the observed reduction is attributable to skipping already-available early stages or to rejecting before numerical work, conditional on the frozen portfolio.

The explicit reject action is also distinct from solver failure. A runtime solver that spends hundreds of FK evaluations before returning failure may be operationally unsuitable for a controller with a nominal 20 ms update interval, although this study did not measure hard deadline compliance. The learned gate recognized the two constructed rejectable strata with high recall, converted most of them into a zero-solve refusal, and retained the same final rejection rate as the robust cascade. This is refusal under a kinematic command contract, not a proof of global unreachability for arbitrary obstacles or dynamics.

## 6.2 Why the negative latency result matters

FEV is an algorithmic proxy, not end-to-end latency. The formal test demonstrates that reducing numerical evaluations does not guarantee a faster system when batch-one model inference is inefficient. Reporting only the FEV result would overstate the engineering claim. Conversely, discarding the method after the eager overhead would conflate algorithm semantics with a removable implementation path. The validation pilot resolves this distinction by preserving outputs and routing while replacing Python dispatch with frozen vectorized parameters and an exact exported ensemble.

The correct evidential sequence is therefore formal negative result, validation-only remediation, locked deployment artifacts, and finally a fresh one-shot test. The optimized validation result is encouraging but cannot retroactively repair the frozen formal test. This sequence is more conservative than selecting a faster backend after examining test outcomes and then reporting it on the same queries.

## 6.3 Interpretation of null ablations

The null single-member, uncertainty, calibration, and no-history operational ablations limit the contribution claim. They do not show that these components are useless in other robots or distribution shifts; rather, this benchmark cannot attribute the primary FEV effect to any one of them. The strongest supported learned mechanism is the calibrated action gate as a whole, especially explicit rejection. Future work should use distribution shifts that create genuine disagreement and branch ambiguity if it aims to establish an ensemble-specific benefit.

The threshold guard further illustrates this point. A Cartesian-step rule cheaply identified many rejectable queries but sacrificed approximately 0.9 percentage points of feasible success relative to the proposed method. The learned gate's value is not merely detecting a large pose change; its portfolio-aware labels encode whether the full verifier can be satisfied by a later stage.

## 6.4 Limitations and next validation steps

First, all formal evidence is kinematic. The URDF model is exact, collision checking is absent, and no controller, contact, torque, acceleration, sensor delay, or hardware uncertainty is simulated. Isaac Lab was the software environment source for robot assets, but the present benchmark is not an Isaac Lab physics experiment. Physics-based controller tracking should be evaluated as a separate locked phase with identical low-level control for all methods.

Second, the point mixture is deliberately stratified and does not estimate natural deployment prevalence. Reported mixture means should not be interpreted as expected field performance without an application-specific query distribution. Third, the models are robot-specific and were trained separately for Panda and UR5e; the study does not demonstrate the cross-kinematic generalization targeted by GGIK [7]. Fourth, the verifier enforces pose, limits, and one-frame velocity only. Self-collision, environment collision, acceleration, jerk, and torque must be added for broader admissibility claims.

Fifth, three training seeds share one runtime test per robot. They demonstrate sensitivity consistency but do not provide six independent test datasets. Sixth, the optimized backend has been evaluated only on validation and only with seed 17. At the time of this draft, deployment packaging has not passed every robot/seed equivalence

gate and no fresh optimized test has been started. The paper is not ready to replace the formal latency conclusion until all six artifacts are sealed and a preregistered `test_v3` passes without threshold or claim-gate changes.

Finally, the gate is reactive to one-step diagnostics. A future system could combine fast history tracking with path-level candidate selection, explicit singularity-domain transition management, collision-aware verification, and calibrated out-of-distribution detection. A particularly testable extension is two-sided abstention: high-confidence portfolio failure would trigger command rejection, whereas model uncertainty or OOD evidence would defer to the complete robust cascade rather than reject. That extension requires new counterfactual action labels, shifted validation sets, and a fresh locked test; none of its benefits are inferred from the present results. Such work should retain the same separation between proposal, selection, refinement, and verification.

# 7 Conclusions

This work reframes online inverse kinematics as kinematically verified allocation of solver effort. Across two robot-specific locked test sets and three training-seed sensitivity runs per robot, a calibrated history-conditioned gate preserved feasible-query success, reduced mean numerical evaluations, rejected futile queries before expensive fallback, and retained the tested descriptive trajectory outcomes relative to an otherwise identical fixed-entry cascade. The conventional-comparator results bound this claim: simpler DLS was faster in eager batch-one timing, whereas the Cartesian-step guard sacrificed feasible and hard-valid success. The common kinematic verifier, not the neural network, remained the authority for command acceptance.

The study also produced a useful negative result: the original eager risk model erased the numerical saving on feasible-query P95 latency, causing the frozen formal claim gate to fail. An exact validation-only export removed most fixed overhead and met the pilot ratios on both robots without changing routing or numerical outcomes, but this result still requires completed packaging and independent optimized testing. The resulting conclusion is not that neural IK replaces numerical IK. Learning predicts where to start, how much work to allocate, and when to refuse; numerical geometry refines, verifies, and provides a bounded fallback.

# Author Contributions

[AUTHOR INPUT NEEDED: use the CRediT taxonomy and assign conceptualization, methodology, software, validation, formal analysis, visualization, writing, supervision, and funding acquisition to named authors.]

# Funding

[AUTHOR INPUT NEEDED: funding agency, grant number, and statement; write “This research received no external funding” only if factually correct.]

# Institutional Review Board Statement

Not applicable; the study used robot kinematic models and synthetic queries and did not involve humans or animals.

# Informed Consent Statement

Not applicable.

# Data Availability Statement

The manuscript source-data CSV files, extraction scripts, configuration files, and hash manifests are included in the reproducibility package. The frozen query-level outputs and model artifacts will be deposited at [AUTHOR INPUT NEEDED: public repository and DOI/accession] before publication. The optimized `test_v3` dataset and results are intentionally absent because that test has not been authorized or run at the time of this draft.

# Acknowledgments

[AUTHOR INPUT NEEDED: acknowledgments, if any.]

## Use of Generative Artificial Intelligence

During manuscript preparation, the authors used OpenAI Codex to assist with language drafting, LaTeX preparation, code review, and reproducible figure generation. All scientific claims and numerical values were checked against frozen experiment outputs; the authors reviewed the resulting text and take full responsibility for the published content.

## Conflicts of Interest

The authors declare no conflict of interest. [AUTHOR INPUT NEEDED: confirm this statement and add any relationships required by the journal.]

## A Per-Run Primary Results

**Table 7:** Exact per-run primary metrics from the frozen formal test. Latency is P95 in milliseconds.

| Robot | Seed | Success fixed/proposed | FEV fixed/proposed | Feasible P95 fixed/proposed | Reject P95 reduction | Traj. fixed/proposed |
|---|---|---|---|---|---|---|
| Panda | 17 | 0.9999/0.9999 | 5.282/3.777 | 3.372/4.444 | 0.9905 | 1.000/1.000 |
| Panda | 29 | 0.9999/0.9999 | 5.279/3.799 | 3.399/4.694 | 0.9906 | 1.000/1.000 |
| Panda | 43 | 0.9999/0.9999 | 5.233/3.764 | 3.571/4.899 | 0.9890 | 1.000/1.000 |
| UR5e | 17 | 1.0000/1.0000 | 5.943/3.950 | 3.136/4.716 | 0.9788 | 0.950/0.975 |
| UR5e | 29 | 1.0000/1.0000 | 5.945/3.937 | 3.169/5.251 | 0.9775 | 0.975/0.975 |
| UR5e | 43 | 1.0000/1.0000 | 5.939/3.924 | 3.155/5.199 | 0.9772 | 0.975/0.975 |

## B Validation-Only Latency Table

**Table 8:** Selected exact-backend feasible-query latency on validation only (750 paired queries per robot, training seed 17).

| Robot | Fixed P50 | Proposed P50 | Fixed P95 | Proposed P95 | Fixed P99 | Proposed P99 | P95 ratio |
|---|---|---|---|---|---|---|---|
| Panda | 1.670 | 1.922 | 3.358 | 2.656 | 4.386 | 4.697 | 0.791 |
| UR5e | 1.554 | 1.807 | 3.090 | 2.705 | 3.340 | 3.315 | 0.875 |

## References

[1] Charles W. Wampler. Manipulator inverse kinematic solutions based on vector formulations and damped least-squares methods. *IEEE Transactions on Systems, Man, and Cybernetics*, 16(1):93–101, 1986. doi: 10.1109/TSMC.1986.289285.

[2] Yoshihiko Nakamura and Hideo Hanafusa. Inverse kinematic solutions with singularity robustness for robot manipulator control. *Journal of Dynamic Systems, Measurement, and Control*, 108(3):163–171, 1986. doi: 10.1115/1.3143764.

[3] Stefano Chiaverini, Bruno Siciliano, and Olav Egeland. Review of the damped least-squares inverse kinematics with experiments on an industrial robot manipulator. *IEEE Transactions on Control Systems Technology*, 2(2): 123–134, 1994. doi: 10.1109/87.294335.

[4] Patrick Beeson and Barrett Ames. TRAC-IK: An open-source library for improved solving of generic inverse kinematics. In *2015 IEEE-RAS 15th International Conference on Humanoid Robots*, pages 928–935, 2015. doi: 10.1109/HUMANOIDS.2015.7363472.

[5] Jacinto Colan, Ana Davila, and Yasuhisa Hasegawa. Variable step sizes for iterative jacobian-based inverse kinematics of robotic manipulators. *IEEE Access*, 12:87909–87922, 2024. doi: 10.1109/ACCESS.2024.3418206.

[6] Barrett Ames, Jeremy Morgan, and George Konidaris. IKFlow: Generating diverse inverse kinematics solutions. *IEEE Robotics and Automation Letters*, 7(3):7177–7184, 2022. doi: 10.1109/LRA.2022.3181374.

[7] Oliver Limoyo, Filip Marić, Matthew Giamou, Petra Alexson, Ivan Petrović, and Jonathan Kelly. Generative graphical inverse kinematics. *IEEE Transactions on Robotics*, 41:1002–1018, 2025. doi: 10.1109/TRO.2024.3521862.

[8] Jeremy Morgan, David Millard, and Gaurav S. Sukhatme. CppFlow: Generative inverse kinematics for efficient and robust cartesian path planning. In *2024 IEEE International Conference on Robotics and Automation*, 2024. doi: 10.1109/ICRA57147.2024.10611724.

[9] Jiahao Yang, Shenhao Yan, Fan Feng, Chengsi Yao, Ge Wang, Zhixin Mai, Yiming Zhao, and Yatong Han. MimicIK: Real-time generative inverse kinematics from teleoperation with fk consistency, 2026. Preprint.

[10] Jan-Gerrit Habekost, Erik Strahl, Philipp Allgeuer, Matthias Kerzel, and Stefan Wermter. CycleIK: Neuro-inspired inverse kinematics. In *Artificial Neural Networks and Machine Learning – ICANN 2023*, pages 457–470. Springer, 2023. doi: 10.1007/978-3-031-44207-0_38.

[11] Meenalochani Jayabalan, Karunamoorthy Loganathan, and Palanikumar Kayaroganam. A novel hybrid ik architecture for robotic arms: Iterative refinement of soft-computing approximations with validation on abb irb-1200 robotic arm. *Machines*, 14(3):292, 2026. doi: 10.3390/machines14030292.

[12] Ali Jlidi, Rabab Benotsmane, and László Kovács. Explainable graph neural networks towards data-driven inverse kinematics in industrial robot motion planning. *Electronics*, 15(14):3071, 2026. doi: 10.3390/electronics15143071.

[13] Xinyi Yuan, Weiwei Wan, and Kensuke Harada. IKSel: Selecting good seed joint values for fast numerical inverse kinematics iterations, 2025.

[14] Daniel Rakita, Bilge Mutlu, and Michael Gleicher. RelaxedIK: Real-time synthesis of accurate and feasible robot arm motion. In *Robotics: Science and Systems XIV*, 2018. doi: 10.15607/RSS.2018.XIV.043.

[15] Alexander J. Elias and John T. Wen. IK-Geo: Unified robot inverse kinematics using subproblem decomposition. *Mechanism and Machine Theory*, 209:105971, 2025. doi: 10.1016/j.mechmachtheory.2025.105971.

[16] Yachen Zhang and Ryo Kikuuwe. Ensuring viability: A QP-based inverse kinematics for handling joint range, velocity and acceleration limits, as well as whole-body collision avoidance. *Journal of Intelligent & Robotic Systems*, 112(1):16, 2026. doi: 10.1007/s10846-025-02335-z.

[17] Chen Xu, Tony Khuong Nguyen, Emma Dixon, Christopher Rodriguez, Patrick Miller, Robert Lee, Paarth Shah, Rares Andrei Ambrus, Haruki Nishimura, and Masha Itkina. Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies. In *Proceedings of Robotics: Science and Systems XXI*, Los Angeles, CA, USA, 2025. doi: 10.15607/RSS.2025.XXI.073.

[18] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[19] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pages 1321–1330. PMLR, 2017.

[20] Justin Carpentier, Guilhem Saurel, Gabriele Buondonno, Joseph Mirabel, Florent Lamiraux, Olivier Stasse, and Nicolas Mansard. The pinocchio c++ library: A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives. In *2019 IEEE/SICE International Symposium on System Integration*, pages 614–619, 2019. doi: 10.1109/SII.2019.8700380.

[21] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[22] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[23] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.